brain/carl_physarum.py — Full Line-by-Line Walkthrough

Length: 257 lines. **Role:** models the arena as an electrical/fluid network and solves for a pressure field between a source and a sink cell, using that field both as a navigation gradient and (in principle) as the substrate for genuine Physarum ("slime mold") tube-thickening dynamics. This is another standalone planning module — not imported by `carl_simulation.py` — and, as the walkthrough below shows, its most distinctive named feature (adaptive tube conductivity) is actually built but then bypassed every time `step()` runs, leaving it functioning in practice as a repeated harmonic-pressure solver (closer to Dijkstra/electrical-network shortest-path) rather than true slime-mold growth.

Lines 1–10: Header and imports

```python
"""
carl_physarum.py — Physarum Polycephalum Slime Mold Path Optimizer
Based on: Tero, Kobayashi & Nakagaki (2007) Science — proven shortest-path co

The maze is a network of tubes. Tubes carrying high flow THICKEN.
Tubes carrying no flow RETRACT. System provably converges to Steiner shortest
"""
import numpy as np
from scipy import sparse
from scipy.sparse.linalg import spsolve
```

The cited paper is real and well-known: Tero et al. (2007) built an actual mathematical model of how the slime mold *Physarum polycephalum* solves maze shortest-path problems by having its protoplasmic tubes physically thicken where flow is high and shrink where flow is low, with the whole network provably converging toward the shortest connecting path. `scipy.sparse` and `spsolve` are used because the "solve for the pressure field" step below is really solving a large linear system, and the vast majority of that system's matrix entries are zero (each grid cell only connects to its 4 immediate neighbors), so a sparse solver is dramatically faster and more memory-efficient than a dense one at any nontrivial grid size.

Lines 12–16: Physarum-model parameters

```python
```

```
MU      = 1.8      # Reinforcement exponent (1 < mu < 2 for convergence)
TAU     = 80.0     # Conductivity adaptation time constant (steps)
D_MIN   = 1e-5     # Minimum conductivity (prevents disconnection)
D_MAX   = 50.0     # Maximum conductivity
```

`MU` (the exponent in the tube-adaptation equation used below) sits in the `1 < μ < 2` range the comment cites as the mathematically proven regime where the network provably converges to the shortest Steiner-tree-like path rather than oscillating or collapsing — this is a real constraint from the cited paper's own analysis, not an arbitrarily chosen number. `D_MIN`/`D_MAX` bound how thin or thick a tube's conductivity can become, with the nonzero floor (`1e-5`, not literally zero) existing specifically so a tube is never *perfectly* disconnected, which would make the network's underlying linear system singular (unsolvable).

`PhysarumMaze.__init__` (lines 26–45)

python

```python
def __init__(self, rows=25, cols=25):
    self.R = rows
    self.C = cols
    self.N = rows * cols
    self.D_h = np.ones((rows, cols-1), dtype=float)
    self.D_v = np.ones((rows-1, cols), dtype=float)
    self.obstacle = np.zeros((rows, cols), dtype=bool)
    self.pressure = np.zeros(self.N, dtype=float)
    self.nav_signal = np.zeros((rows, cols), dtype=float)
    self._step_count = 0
```

Models the same 25×25 grid resolution used elsewhere in this repository's alternate "cognitive map" tooling (matching `astar.py`'s `MAP_RES=25`). `D_h` (horizontal tube conductivities, one entry per pair of horizontally-adjacent cells) and `D_v` (vertical) both start at a uniform conductivity of `1.0` — every possible tube in the grid starts equally open, with no path yet preferred. `obstacle` is a boolean mask over cells, refreshed from the cognitive map each update. `pressure` is the flat (length-`N`) solution vector the linear solver will fill in. `nav_signal` is the final, per-cell output this whole class exists to produce: a scalar at every grid cell indicating how strongly the current flow points toward the sink from there.

Lines 47–52: Grid index helpers

python

```
def _idx(self, i, j):
    return int(i) * self.C + int(j)

def _pos(self, idx):
    return idx // self.C, idx % self.C
```

Standard 2D-to-1D flattening (row-major order) and its inverse — needed because the sparse linear solver operates on a flat vector of `N = rows × cols` unknowns (one pressure value per grid cell), not a 2D array.

Lines 54–57: `update_obstacles()`

python

```
def update_obstacles(self, CM_danger, danger_thresh=0.7):
    self.obstacle = CM_danger > danger_thresh
```

Takes an externally-supplied danger channel (the same kind of `CM[:,:,0]` array `astar.py` reads) and thresholds it into a hard boolean obstacle mask at `0.7` — notably a *lower*, more permissive threshold than `astar.py`'s `0.85` for its own (unused) wall mask, meaning this module treats moderately dangerous cells as fully impassable rather than merely costly, a stricter safety posture than the cost-based approach in `astar.py`.

`_build_laplacian()` (lines 60–97) — constructing the electrical network matrix

python

```
def _build_laplacian(self):
    data, row_idx, col_idx = [], [], []
    diag = np.zeros(self.N, dtype=float)

    for i in range(self.R):
        for j in range(self.C):
            n = self._idx(i, j)
            if self.obstacle[i, j]:
                data.append(1e-6)
                row_idx.append(n); col_idx.append(n)
                continue
```

This builds what's known in circuit theory / spectral graph theory as a (weighted) graph Laplacian — the matrix that, once solved with fixed boundary pressures, tells you the steady-state pressure/voltage at every node of a resistor/tube network. Obstacle cells are handled by giving them only a tiny self-connection (`1e-6` on their own diagonal entry)

and `continue`-ing past any neighbor-connection logic — this effectively isolates them from the rest of the network (they can't carry any meaningful flow) while still keeping the overall matrix non-singular (a completely disconnected/zero row would make the linear system unsolvable).

python

```
            if j + 1 < self.C and not self.obstacle[i, j+1]:
                m = self._idx(i, j+1)
                d = float(self.D_h[i, j])
                data.append(-d); row_idx.append(n); col_idx.append(m)
                data.append(-d); row_idx.append(m); col_idx.append(n)
                diag[n] += d; diag[m] += d

            if i + 1 < self.R and not self.obstacle[i+1, j]:
                m = self._idx(i+1, j)
                d = float(self.D_v[i, j])
                data.append(-d); row_idx.append(n); col_idx.append(m)
                data.append(-d); row_idx.append(m); col_idx.append(n)
                diag[n] += d; diag[m] += d
```

For every non-obstacle cell, check its right neighbor and its neighbor below (checking only these two directions per cell, rather than all four, is sufficient because each pair gets recorded symmetrically in both directions at once — `n→m` and `m→n` — so by the time the double loop finishes every cell's left/up connections have already been filled in when that neighbor was the "current" cell earlier in the scan). Each connected pair gets an off-diagonal entry of `-conductivity` in both `(n,m)` and `(m,n)` positions (this symmetric negative-off-diagonal, positive-diagonal structure is exactly the standard graph Laplacian: `L = D - W`, degree matrix minus weighted adjacency), and both nodes' running diagonal totals accumulate that same conductivity — the diagonal of a Laplacian always equals the sum of a node's own edge weights.

python

```
        for n in range(self.N):
            if diag[n] < 1e-10:
                diag[n] = 1e-6
            data.append(diag[n])
            row_idx.append(n); col_idx.append(n)

        L = sparse.csr_matrix((data, (row_idx, col_idx)), shape=(self.N, self.N))
        return L
```

`_solve_pressure()` (lines 100–115) — applying boundary conditions and solving

```python
def _solve_pressure(self, src_idx, snk_idx):
    L = self._build_laplacian()
    rhs = np.zeros(self.N, dtype=float)

    L_mod = L.tolil()
    L_mod[src_idx, :] = 0; L_mod[src_idx, src_idx] = 1.0; rhs[src_idx] = 1.0
    L_mod[snk_idx, :] = 0; L_mod[snk_idx, snk_idx] = 1.0; rhs[snk_idx] = 0.0

    try:
        p = spsolve(L_mod.tocsr(), rhs)
        if np.any(np.isnan(p)): p = np.zeros(self.N)
    except Exception:
        p = np.zeros(self.N)

    self.pressure = np.clip(p, 0.0, 1.0)
```

This is a classic Dirichlet boundary-condition trick for solving `Laplacian × pressure = 0` (the physical equation governing steady-state flow with no internal sources/sinks) while *forcing* specific known values at two nodes: it wipes out the source node's entire row and replaces it with an identity-style single `1.0` on the diagonal, then sets the corresponding right-hand-side entry to `1.0` — this makes that row of the linear system literally say "pressure at source = 1.0" instead of the Laplacian's normal flow-balance equation. The same trick pins the sink node's pressure to exactly `0.0`. `.tolil()` (list-of-lists format) is used for these row-overwrites because sparse CSR matrices are inefficient to modify row-by-row, while LIL format is built for exactly that; `.tocsr()` converts back before the actual solve, since CSR is what `spsolve` is optimized for. The `try/except` and `NaN` check are defensive: an ill-conditioned or degenerate network (e.g., source and sink both fully walled off from each other by obstacles) could produce a numerically unstable or unsolvable system, in which case the code falls back to an all-zero pressure field rather than crashing or propagating garbage numbers. The final `np.clip(p, 0.0, 1.0)` both bounds the result to the physically sensible `[0,1]` range (matching the fixed boundary values) and guards against small numerical overshoot from the solver.

`_update_conductivities()` (lines 118–142) — the real Physarum adaptation rule

```python
def _update_conductivities(self):
    p = self.pressure.reshape(self.R, self.C)
    p_left  = p[:, :-1]
    p_right = p[:, 1:]
    Q_h = np.abs(self.D_h * (p_left - p_right))
    dD_h = (Q_h**MU - self.D_h) / TAU
    self.D_h = np.clip(self.D_h + dD_h, D_MIN, D_MAX)
```

This is the genuine mathematical model from the cited Tero et al. paper: flow through a tube (`Q`) is conductivity times the pressure difference across it (Ohm's-law-like: `Q = D × Δp`), computed here with vectorized NumPy slicing (`p[:, :-1]` and `p[:, 1:]` are the "left" and "right" pressure grids offset by one column, so subtracting them gives every horizontal pressure gradient at once without an explicit loop). The actual adaptation equation, `dD/dt = (Q^μ - D) / τ`, is the paper's core finding: a tube's conductivity relaxes, over the `TAU=80`-step time constant, toward `Q^μ` — meaning tubes carrying strong flow (`Q` large) get pulled toward a much larger equilibrium conductivity (especially with `μ=1.8`, since raising a large number to a power close to 2 amplifies it further), while tubes with near-zero flow relax toward near-zero conductivity — literally "high flow thickens, no flow retracts," as the file's own header states. `np.clip` enforces the `D_MIN`/`D_MAX` bounds set as globals.

```python
    for i in range(self.R):
        for j in range(self.C-1):
            if self.obstacle[i,j] or self.obstacle[i,j+1]:
                self.D_h[i,j] = D_MIN
```

Even after the adaptive update, any tube touching an obstacle cell on either end gets forced back down to the conductivity floor — the same defensive pattern applies to the vertical tubes just below it. This is a straightforward loop rather than a vectorized operation, which is fine at 25×25 scale but would become a real performance bottleneck at a much larger grid resolution.

`_compute_nav_signal()` (lines 145–164) — turning pressure into a steering signal

```python
```

```
def _compute_nav_signal(self):
    p = self.pressure.reshape(self.R, self.C)
    sig = np.zeros((self.R, self.C), dtype=float)

    sig[:, :-1] += self.D_h * np.maximum(0, p[:, :-1] - p[:, 1:])
    sig[:, 1:]  += self.D_h * np.maximum(0, p[:, 1:] - p[:, :-1])
    sig[:-1, :] += self.D_v * np.maximum(0, p[:-1, :] - p[1:, :])
    sig[1:,  :] += self.D_v * np.maximum(0, p[1:,  :] - p[:-1, :])

    mx = sig.max()
    if mx > 1e-6: sig /= mx
    self.nav_signal = sig
```

For every cell, this accumulates a signal strength from each of its (up to 4) neighboring edges, but only counting the *outflow* direction (`np.maximum(0, ...)` zeroes out any negative pressure-drop term — a cell only accumulates signal from edges where pressure is flowing *away* from it toward a lower-pressure neighbor, not the reverse). Because the sink was pinned to pressure `0.0` and the source to `1.0`, pressure naturally decreases along any real flow path toward the sink — so a cell sitting on a strong-flow tube ends up with a high `nav_signal`, and that signal is inherently directional (it's built from *which side* has lower pressure), which is exactly what a downstream steering controller needs. The final normalization by the grid-wide maximum rescales everything into a comparable `[0,1]`-ish range regardless of how large the raw conductivities happen to be.

`step()` (lines 167–202) — the actual per-tick update, and where adaptation gets skipped

```
python
```

```
def step(self, src_cell, snk_cell, CM_danger=None, every=1):
    self._step_count += 1
    if CM_danger is not None:
        self.update_obstacles(CM_danger)

    si = self._idx(*src_cell)
    sk = self._idx(*snk_cell)
    if si == sk:
        self.nav_signal[:] = 0.0
        return

    self._solve_pressure(si, sk)

    self.D_h[:] = 1.0
    self.D_v[:] = 1.0
    for i in range(self.R):
        for j in range(self.C-1):
            if self.obstacle[i, j] or self.obstacle[i, j+1]:
                self.D_h[i, j] = D_MIN
    for i in range(self.R-1):
        for j in range(self.C):
            if self.obstacle[i, j] or self.obstacle[i+1, j]:
                self.D_v[i, j] = D_MIN

    self._compute_nav_signal()
```

This is the most important thing to understand about the file: `_update_conductivities()` — the method that implements the actual, named, cited slime-mold thickening dynamics — is **never called from `step()`**. Instead, every single call to `step()` unconditionally resets `D_h` and `D_v` back to a uniform `1.0` (line "`self.D_h[:] = 1.0`") before re-applying only the obstacle mask, then immediately solves for pressure fresh on this uniform-conductivity network. The inline comment ("Keep conductivities active to prevent 'dead-tube' lock-in") explains the intent — likely a deliberate choice to keep the network fully responsive to a *moving* source/sink (a live robot chasing a live target) rather than letting the network's memory of past flow patterns bias it toward previously-good routes, which would be a liability if the target moves. The practical consequence: this class runs a fresh electrical-network / harmonic shortest-path solve every tick — mathematically closely related to Dijkstra's algorithm on a resistance-weighted graph — rather than exhibiting actual adaptive tube growth over time. `_update_conductivities()` is fully implemented, tested-looking code that currently has no caller anywhere in this file, the same "built but not wired up" pattern already seen in `carl_endocrine.py`'s `surge_ach()` and `get_reward_prediction_error()`.

`best_move()` (lines 205–226) — greedy steepest-descent stepping

python

```python
def best_move(self, cur_cell, src_cell, snk_cell):
    i, j = int(cur_cell[0]), int(cur_cell[1])
    p = self.pressure.reshape(self.R, self.C)
    candidates = []
    for di, dj in [(-1,0), (1,0), (0,-1), (0,1)]:
        ni, nj = i+di, j+dj
        if 0 <= ni < self.R and 0 <= nj < self.C:
            if not self.obstacle[ni, nj]:
                if p[ni, nj] < p[i, j] + 1e-5:
                    candidates.append(((ni, nj), p[i, j] - p[ni, nj]))
    if not candidates:
        return cur_cell
    return max(candidates, key=lambda x: x[1])[0]
```

A simple greedy hill-climb: from the current cell, look at the 4 orthogonal neighbors, discard obstacles and any neighbor whose pressure isn't at least as low as the current cell's (the small `1e-5` tolerance allows equal-pressure moves, avoiding getting stuck purely from floating-point noise), and among the valid candidates, pick whichever has the *largest* pressure drop — the steepest local descent. The comment's claim ("guarantees zero local minima") follows directly from the mathematical fact that pressure in this kind of harmonic (Laplace-equation) field has no interior local minima away from the fixed source/sink boundary conditions — any interior cell's pressure is exactly the (conductivity-weighted) average of its neighbors, so there's always at least one neighbor with lower pressure to step toward, all the way until reaching the sink. If somehow every neighbor is invalid (a fully boxed-in cell), it simply returns the current cell unchanged (a no-op "stay put").

`get_heading()` (lines 229–251) — world-coordinate wrapper

python

```
def get_heading(self, xw, yw, goal_xw, goal_yw,
                MAP_XMIN, MAP_XMAX, MAP_YMIN, MAP_YMAX):
    def to_grid(x, y):
        gi = int(np.clip((x - MAP_XMIN)/(MAP_XMAX - MAP_XMIN)*self.R, 0, self
        gj = int(np.clip((y - MAP_YMIN)/(MAP_YMAX - MAP_YMIN)*self.C, 0, self
        return gi, gj

    cur  = to_grid(xw, yw)
    snk  = to_grid(goal_xw, goal_yw)
    best = self.best_move(cur, cur, snk)
    if best == cur:
        import math
        return math.atan2(goal_yw - yw, goal_xw - xw)

    bx = MAP_XMIN + (best[0] + 0.5) / self.R * (MAP_XMAX - MAP_XMIN)
    by = MAP_YMIN + (best[1] + 0.5) / self.C * (MAP_YMAX - MAP_YMIN)
    import math
    return math.atan2(by - yw, bx - xw)
```

The same world-to-grid rescaling seen in `astar.py`'s `_map_cell()`, packaged here as a local closure `to_grid`. It converts the current and goal world positions to grid cells, asks `best_move()` for the single best next step, and — if that step is a genuinely different cell — converts it back to world coordinates and returns the compass heading (`atan2`) pointing toward it. If `best_move()` returned the same cell unchanged (meaning the local pressure gradient offered no improving neighbor, e.g., very close to the goal or in a boxed-in spot), it falls back to a direct straight-line heading toward the raw goal coordinates rather than getting stuck reporting a zero/undefined direction. `import math` appears twice, once in each branch, rather than once at the top of the file with `numpy` and `scipy` — functionally harmless (Python caches modules after the first import, so a second `import math` elsewhere is nearly free), but an inconsistent style choice compared to how imports are handled everywhere else in this file.

`path_strength()` (lines 253–256)

```python
def path_strength(self):
    return float(np.percentile(np.concatenate([
        self.D_h.flatten(), self.D_v.flatten()]), 90))
```

A single diagnostic scalar: flattens both conductivity arrays into one list and reports the 90th percentile value — "how strong is the network's best-connected tenth of tubes," a rough one-number summary of how strongly the current path stands out from the rest of

the grid. Given that `step()` resets both conductivity arrays to a uniform `1.0` (aside from obstacle-zeroing) before this could ever be called meaningfully, in the file's current wiring this metric would almost always report a value very close to `1.0` regardless of how clear or contested any particular route is — another symptom of the unused adaptation logic.

Summary

A mathematically real, correctly-implemented harmonic pressure-field solver with a genuinely faithful (if currently dormant) implementation of the Tero et al. Physarum tube-adaptation equation sitting right alongside it. As actually wired in `step()`, though, the class behaves as a fresh electrical-network shortest-path solve every tick rather than an adapting slime-mold network — a deliberate-looking trade-off (favoring responsiveness to a moving target over path memory), but one that means `_update_conductivities()` and `path_strength()` are currently vestigial rather than functionally active, similar in spirit to the unused `surge_ach()`/`get_reward_prediction_error()` gaps already documented in `carl_endocrine.py`.